



Las Americas Institute of Technology

INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS (ITLA)

“SUSTENTANTE”

Josué Aybar Paulino

“MATRÍCULA”

2026-1965

“CARRERA”

Inteligencia Artificial

“MATERIA”

Introducción a la Inteligencia Artificial

“DOCENTE”

Luis Bessewell Feliz

“TEMA”

Análisis y documentación del código fuente de una arquitectura neuronal Transformer: estudio de los archivos model.py, train.py, sample.py del proyecto nanoGPT.

“FECHA”

20/08/2026

“CIUDAD”

Santo Domingo, Distrito Nacional

Índice

Antecedentes.....	pág 3
Introducción.....	pág 4
DESCRIPCIÓN GENERAL DEL ARCHIVO NANO GPT.....	pág 5
1.1 El Transformer utilizado en nanoGPT.....	pág 5
1.2 La función del archivo model.py dentro de nanoGPT.....	pág 5
1.3 Lógica del funcionamiento del Trasformer nanoGPT.....	pág 6
ESTRUCTURA DEL CEREBRO TRANSFORMER.....	pág 7
2.1 Clase LayerNorm nn.module (normalización de datos).....	pág 7
2.2 Diagrama de flujo (LAYERNORM).....	pág 8
2.3 Clase CausalSelfAttention nn.module (Auto Atención).....	pág 9
2.4 Diagrama de flujo (CAUSALSELFATTENTION).....	pág 12
2.5 Clase MLP nn.module (Perceptrón Multicapa).....	pág 13
2.6 Diagrama de flujo (MLP).....	pág 14
2.7 Clase block nn.module (bloque Transformer).....	pág 15
2.8 Diagrama de flujo (BLOCK).....	pág 16
2.9 Clase GPT (config) configuración.....	pág 17
2.10 Diagrama de flujo (GPT CONFIG).....	pág 18
2.11 Clase GPT (nn.Modulo).....	pág 19
2.12 Diagrama de flujo (GPT MODULE).....	pág 21
2.13 Diagrama flujo completo de (MODULE.PY).....	pág 22
CICLO DE ENTRENAMIENTO DE APRENDIZAJE.....	pág 23
3.1 Descripción.....	pág 23
3.2 Diagrama flujo completo de (TRAIN.PY).....	pág26
EL GENERADOR DE TEXTO.....	pág 27
4.1 Descripción.....	pág 27
4.2 Diagrama flujo completo de (SAMPLE.PY).....	pág 28
DIAGRAMA COMPLETO DE NANO GPT.....	pág 29
MODIFICACIONES DE VISUALIZACIÓN.....	pág 30
DOCUMENTACIÓN DESCRIPTIVA DEL PROYECTO.....	pág 32
CONCLUSIÓN.....	pág 35
BIBLIOGRAFÍA.....	pág 36

Antecedentes

El ser humano siempre se ha inclinado hacia la mejora constante de las cosas que le rodean: herramientas, artículos, oficios, trabajo; y el proceso de automatización de tareas no iba a ser diferente. Durante las décadas de los 80s y 90s se desarrollaron las famosas **Redes Neuronales Recurrentes (RNR)**, que se utilizaban para realizar traducciones de texto analizando las últimas palabras de la oración para inferir la siguiente.

En principio fue revolucionario, pero ya en el siglo XXI con el aumento de la documentación disponible, las necesidades de cómputo incrementaron y comenzaron a dificultar el entrenamiento de este tipo de red, debido a que procesaba cada palabra de forma secuencial y, por consiguiente, lenta; olvidando las palabras iniciales y perdiendo el contexto del enunciado original que debía traducir.

Como respuesta a esta problemática, un equipo de investigadores de Google: **Ashish Vaswani, Noam Shazeer, Niki Parmar**, (junto a otros 5 profesionales), comenzaron a investigar de qué manera una red neuronal podía traducir texto rápidamente y que tuviera sentido, es cuando finalmente, en 2017, publican el artículo de investigación “**Attention Is All You Need**” donde se explicó el diseño del mecanismo llamado **Self-Attention (Auto Atención)**, que permitía al modelo calcular la relación entre los diferentes tokens de una secuencia y asignar distintos niveles de importancia mediante los vectores Consulta (Q), Clave (K) y Valor (V). para determinar, qué elementos de una oración, eran más importantes para entender el contexto y poder predecir el siguiente término de la oración de forma rápida y lógica. Es cuando los ingenieros deciden que no necesitan las RNR, en su lugar, las sustituyen por el nuevo tipo de arquitectura neuronal llamada **Transformer**.

Introducción

En este informe de investigación se tomará como objeto de estudio y análisis el código fuente de una arquitectura de red neuronal inspirada en Transformer del documento original **“Attention is All You Need”**, específicamente los archivos **“model.py, train.py y sample.py” del proyecto nanoGPT**, autoría del ingeniero e investigador en inteligencia artificial, **Andrej Karpathy**, quien desarrolló **nanoGPT** como una implementación educativa de modelos GPT basados en Transformer; por considerarla de alto valor fundacional para entender la Inteligencia Digital moderna.

El objetivo principal se encamina a desglosar las partes más importantes del Transformer, para ello, se explicará el código de principio a fin, estableciendo una división por clases, indicando y explicando las funciones que se ejecutan y el propósito específico de cada bloque. Se explicará cómo ocurre el proceso de aprendizaje de este modelo Transformer, así como su posterior generación de texto.

Se presentarán diagramas de flujo por cada clase y etapa desde la carga de los datos hasta la generación de texto, de manera que se facilite el entendimiento de cómo funciona cada una por separado, y se mostrará un diagrama completo para mayor comprensión del flujo de trabajo del código fuente nanoGPT. Finalmente, se mostrarán las modificaciones orientadas a visualizar el funcionamiento interno del corazón del Transformer y cómo distribuye sus probabilidades entre los tokens.

Se plantea como objetivo secundario, demostrar por qué esta arquitectura de red neuronal ha revolucionado la Inteligencia Artificial Generativa, específicamente el Procesamiento del Lenguaje Natural (PLN) que ha permitido el desarrollo de los Grandes Modelos del Lenguaje (LLMs), desde 2017 hasta nuestros días.

1. DESCRIPCIÓN GENERAL DEL ARCHIVO NANO GPT.

1.1 El Transformer utilizado en nanoGPT.

NanoGPT implementa una arquitectura basada en un **Transformer de tipo Decoder-only (solo decodificador)**, inspirada en los modelos GPT. A diferencia del Transformer original presentado en el artículo “Attention Is All You Need”, que estaba compuesto por un codificador (Encoder) y un decodificador (Decoder) para tareas como traducción automática, los modelos GPT utilizan únicamente la parte del decodificador, diseñada de esta manera para la generación autoregresiva de texto, donde el modelo recibe una secuencia de tokens anteriores y aprende a predecir el siguiente token, repitiendo este patrón en forma de bucle, para seguir completando la idea de forma automática. Gracias a este enfoque, nanoGPT es capaz de producir texto siguiendo el mismo principio de funcionamiento de los modelos GPT, aunque en una implementación simplificada y de menor escala, concebida con fines educativos y de experimentación.

1.2. La función del archivo model.py dentro de nanoGPT.

El archivo model.py contiene la definición de la arquitectura neuronal principal utilizada por nanoGPT. En este módulo se encuentran implementadas las clases encargadas de construir el modelo GPT, incluyendo la configuración del modelo (GPTconfig), las capas de normalización (LayerNorm), los mecanismos de atención causal (CausalSelfAttention), las redes neuronales de alimentación directa (Multi-layer Perceptron), los bloques Transformer y la integración final del modelo completo (GPT module).

1.3 Lógica del funcionamiento del Trasformer nanoGPT.

El proceso inicia cuando una secuencia de tokens es convertida en vectores mediante una capa de embebido (el proceso de asignar un valor ID a cada token para identificarlo), a los cuales se añade información sobre la posición de cada token dentro de la secuencia.

Posteriormente, estos vectores atraviesan una serie de bloques Transformer, donde el mecanismo de (**Auto Atención**), identifica las relaciones más importantes entre los tokens y una red (**de propagación hacia adelante**) refina la información obtenida.

Finalmente, una capa lineal calcula la probabilidad del siguiente token. Este token se incorpora a la secuencia y el proceso se repite de forma iterativa para generar texto de manera continua.

En términos generales, el proceso es el siguiente:

1. El texto de entrada se divide en tokens.
2. Cada token se convierte en un embedding (vector numérico).
3. Se añade la codificación posicional para que el modelo conozca el orden de los tokens.
4. La secuencia atraviesa varios bloques Transformer.

En cada bloque:

- **Auto Atención** analiza únicamente los tokens anteriores.
 - **Feed Forward Network** transforma las representaciones obtenidas.
 - Se aplican conexiones residuales y normalización para estabilizar el entrenamiento.
5. Finalmente, una capa lineal produce la probabilidad del siguiente token.
 6. Se selecciona un token y el proceso se repite hasta completar el texto.

2. ESTRUCTURA DEL CEREBRO TRANSFORMER (MODEL.PY).

2.1 Clase LayerNorm nn.module (normalización de datos):

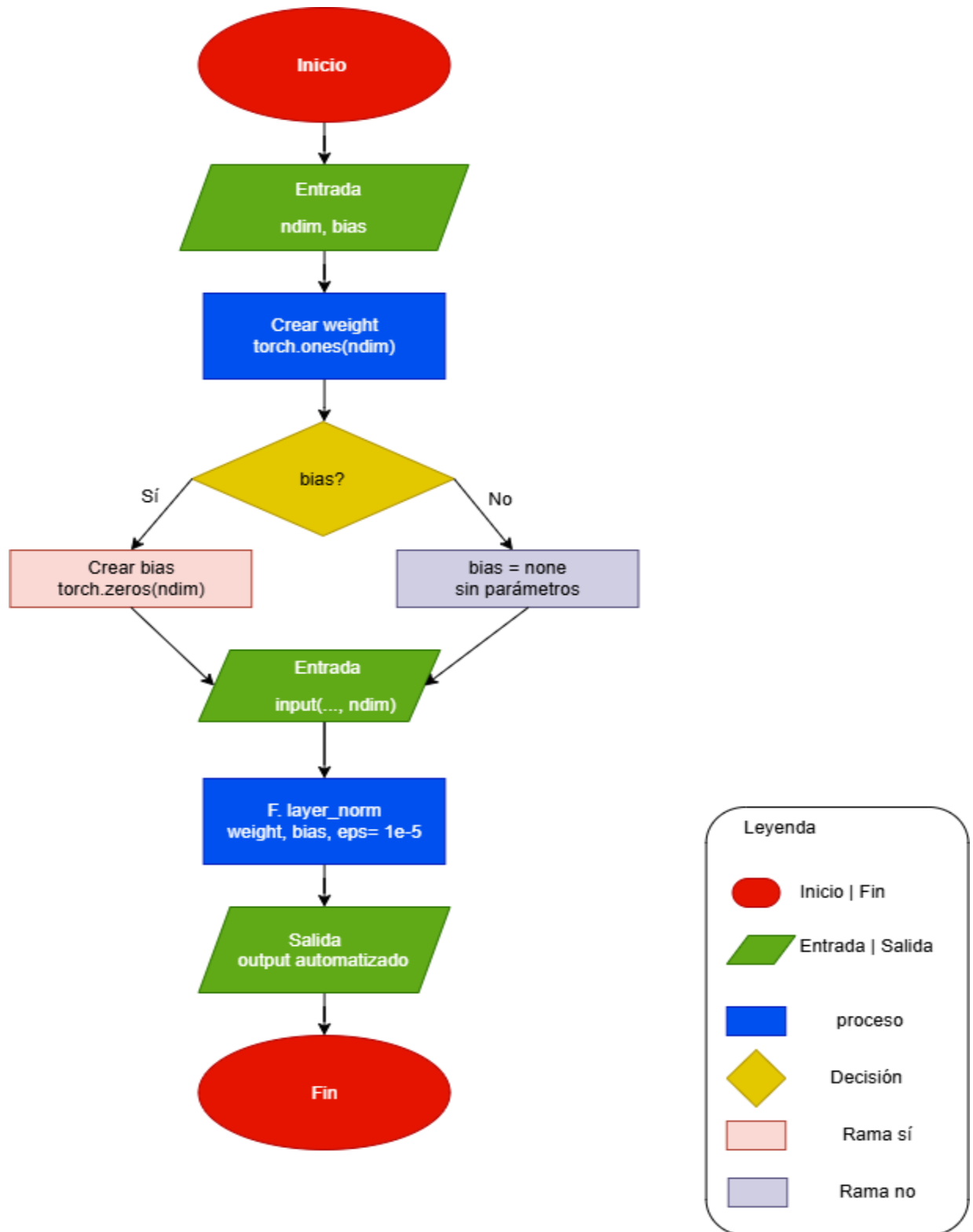
Es la clase que cumple la función de normalizar los valores internos que circulan dentro de la red neuronal, evitando que las activaciones tengan escalas demasiado grandes o pequeñas. Esto ayuda a que los bloques Transformer puedan entrenarse y procesar la información de forma más estable.

Dentro de esta clase se ejecutan las siguientes funciones:

- **La función `__init__`:** se encarga de crear los parámetros entrenables de la capa, como pesos y sesgos, utilizando la cantidad de dimensiones del embebido (ndim).
- **La función `super()`:** inicializa la clase base nn.Module de la librería de PyTorch, permitiendo que esta capa sea reconocida como un componente entrenable del modelo.
- **La función `forward()`:** recibe las representaciones numéricas de los tokens y aplica la operación de normalización mediante la capa de normalización, devolviendo una versión ajustada de los datos para continuar su procesamiento dentro del Transformer.

A continuación, se presenta un diagrama de flujo que visualiza la operación de esta clase:

2.2 DIAGRAMA DE FLUJO (LAYERNORM):



2.3 Clase CausalSelfAttention nn.module (Auto Atención):

Esta es la clase más importante de todo el archivo model.py nanoGPT, porque funciona como el centro de comando ya que permite a los bloques Transformers entender las relaciones entre los tokens y que éstos puedan a su vez, interpretar la relación de los tokens anteriores de la secuencia, esto permite entender el contexto general del texto y ayuda a predecir cuál es la palabra más oportuna para completar la idea inicial. Es lo que hace diferente a los Transformers con respecto a las demás redes neuronales.

Dentro de esta clase se ejecutan estas 2 funciones:

La función `__init__`: que inicia y define las capas para ejecutar la atención.

- **`super().__init__()`:** inicializa la clase como módulo de Pytorch.
- **`assert config.n_embd % config.n_head == 0`:** Verifica que la dimensión del embedding pueda dividirse entre las cabezas de atención.
- **`self.c_attn`:** Genera los vectores **(Q)**, **(K)** y **(V)** de la atención.
- **`self.c_proj`:** Combina la información obtenida por todas las cabezas de atención.
- **`self.flash`:** Determina si se utilizará Flash Attention o el cálculo manual.
- **`Self.bias (máscara causal)`:** Impide que el modelo observe tokens futuros durante la generación.

La función forward (): define el flujo de procesamiento de los datos a través de la clase.

- **B, T, C = x.size():** Obtiene las dimensiones de entrada: lote, secuencia y embedding.
- **self.c_attn(x).split():** Divide la proyección en **Q, K y V**.
- **view() y transpose():** Organiza los datos para trabajar con múltiples cabezas de atención.
- **scaled_dot_product_attention() o cálculo manual:** Ejecuta el mecanismo de atención entre tokens.
- **Máscara causal + Softmax:** Controla qué información puede utilizar cada token y calcula los pesos de importancia.
- **c_proj(y):** Reúne las cabezas y genera la salida final del módulo.

Con respecto a las representaciones (Q, K y V), se procede a realizar una explicación más detallada:

Vector	Pregunta que responde	Función
Query (Q) \ Consulta	¿Qué necesito encontrar?	Busca relaciones relevantes.
Key (K) \ Clave	¿Qué información tengo?	Permite comparar con otras palabras.
Value (V) \ Valor	¿Qué información apporto?	Entrega el contenido que será agregado.

En esta clase, el Transformer ejecuta la siguiente fórmula matemática detrás del framework de Pytorch, para analizar tres representaciones diferentes de un mismo token:

$$Attention(Q, K, V) = Softmax \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Primero, se ejecuta la multiplicación entre **Q** y **K** ^T con el propósito de comparar un token con otro y entender cuál de ellos tiene mayor similitud.

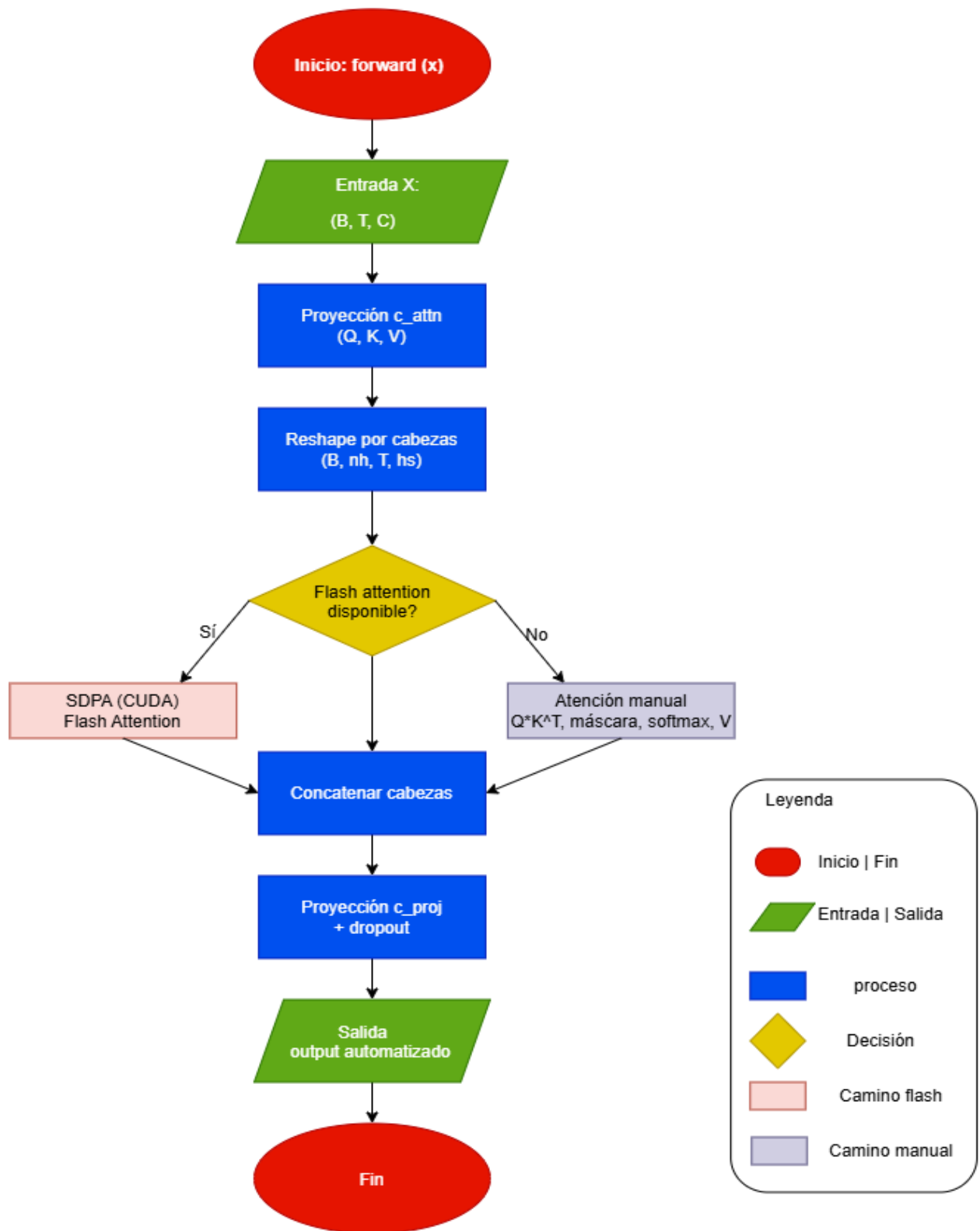
Luego, el producto de la multiplicación anterior se divide entre la raíz cuadrada de la dimensión de la cabeza de atención. Que en nanoGPT: dk=64 porque este código fue configurado con 768 dimensiones, con 12 cabezas multi-atención por bloque Transformer. De manera que, cada una de esas 12 cabezas recibe 3 vectores de 64 dimensiones.

Después, se activa la función softmax, la cual transforma los valores de similitud obtenidos entre Q y K en pesos de atención normalizados. Estos valores indican el nivel de importancia que cada token tiene sobre los demás dentro de la secuencia.

Finalmente, los pesos calculados se aplican sobre los vectores **V**. Esto permite obtener una nueva representación del token, pero más enriquecida con información del contexto completo.

A continuación, se presenta un diagrama de flujo que visualiza la operación de esta clase:

2.4 DIAGRAMA DE FLUJO (CAUSALSELFATTENTION):



2.5 Clase MLP nn.module (Perceptrón Multicapa):

La clase **MLP (Perceptrón Multicapa)** implementa la red neuronal de alimentación directa del bloque Transformer. Su función es transformar la representación contextual obtenida por CausalSelfAttention. La capa `c_fc` expande la dimensión del embedding cuatro veces, permitiendo al modelo trabajar con un espacio de mayor capacidad. Luego, la función de activación GELU introduce no linealidad, `c_proj` reduce nuevamente la representación a la dimensión original y finalmente Dropout aplica regularización para mejorar la capacidad de generalización del modelo.

Dentro de esta clase se ejecutan estas funciones:

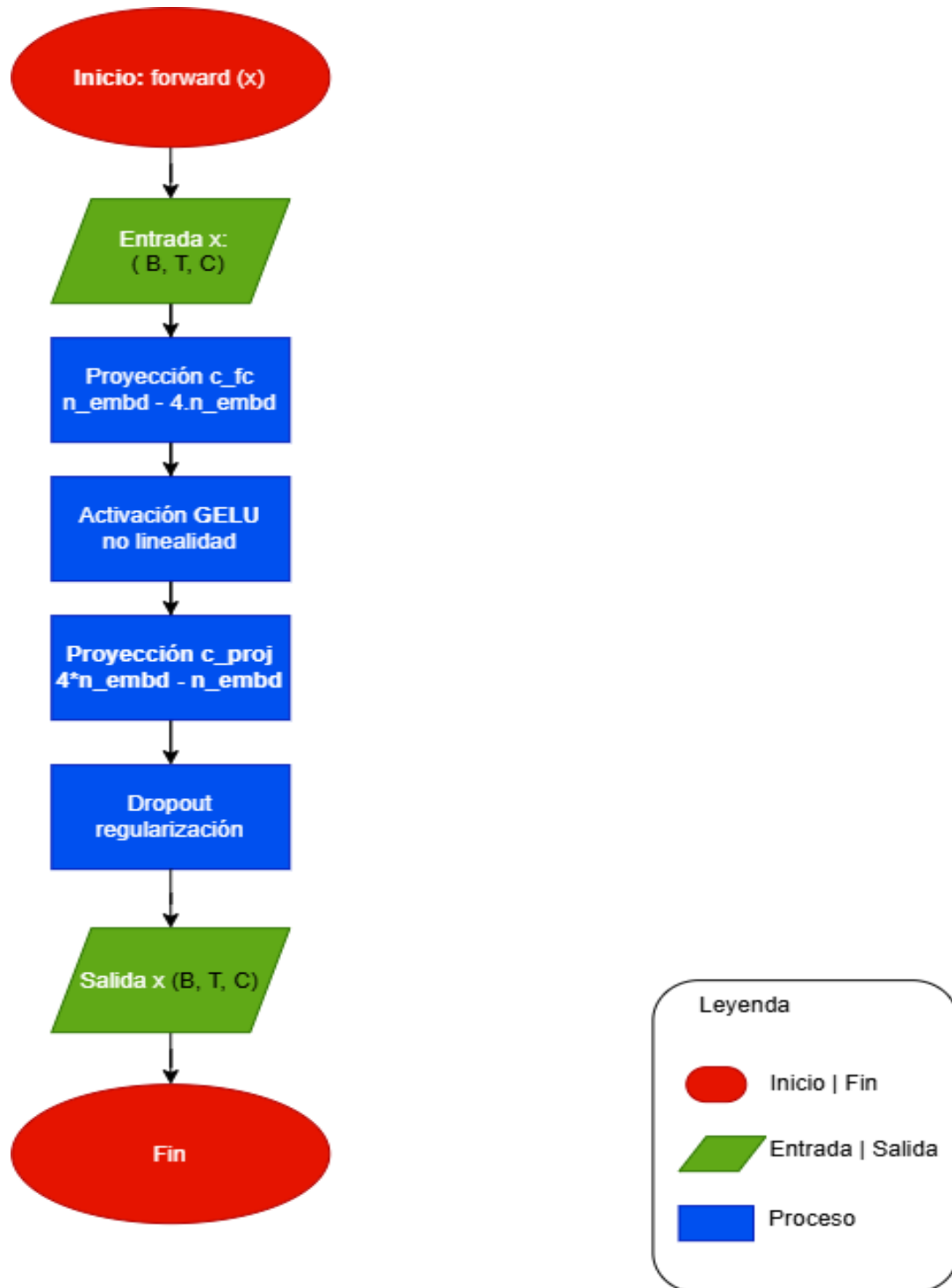
La función `__init__`: para inicializar todas las capas que usará el perceptrón durante su entrenamiento.

- **`c_fc`:** Primera capa lineal que expande el embebido de `n_embd` a $4*n_embd$.
- **GELU:** Función de activación no lineal aplicada después de la expansión.
- **`c_proj`:** Segunda capa lineal que reduce la dimensión de $4*n_embd$ nuevamente a `n_embd`.

La función `forward ()`: Ejecuta la secuencia expansión → activación → reducción → regularización.

A continuación, se presenta un diagrama de flujo que visualiza la operación de esta clase:

2.6 DIAGRAMA DE FLUJO (MLP):



2.7 Clase **block nn.module (bloque Transformer)**:

Representa una unidad básica de la arquitectura Transformer utilizada en NanoGPT. Cada bloque combina dos procesos fundamentales: el mecanismo de Auto Atención, encargado de analizar la relación entre los tokens de la secuencia, y la red de Perceptrón Multicapa, encargada de transformar y enriquecer la información obtenida.

Dentro de esta clase se ejecutan estas funciones:

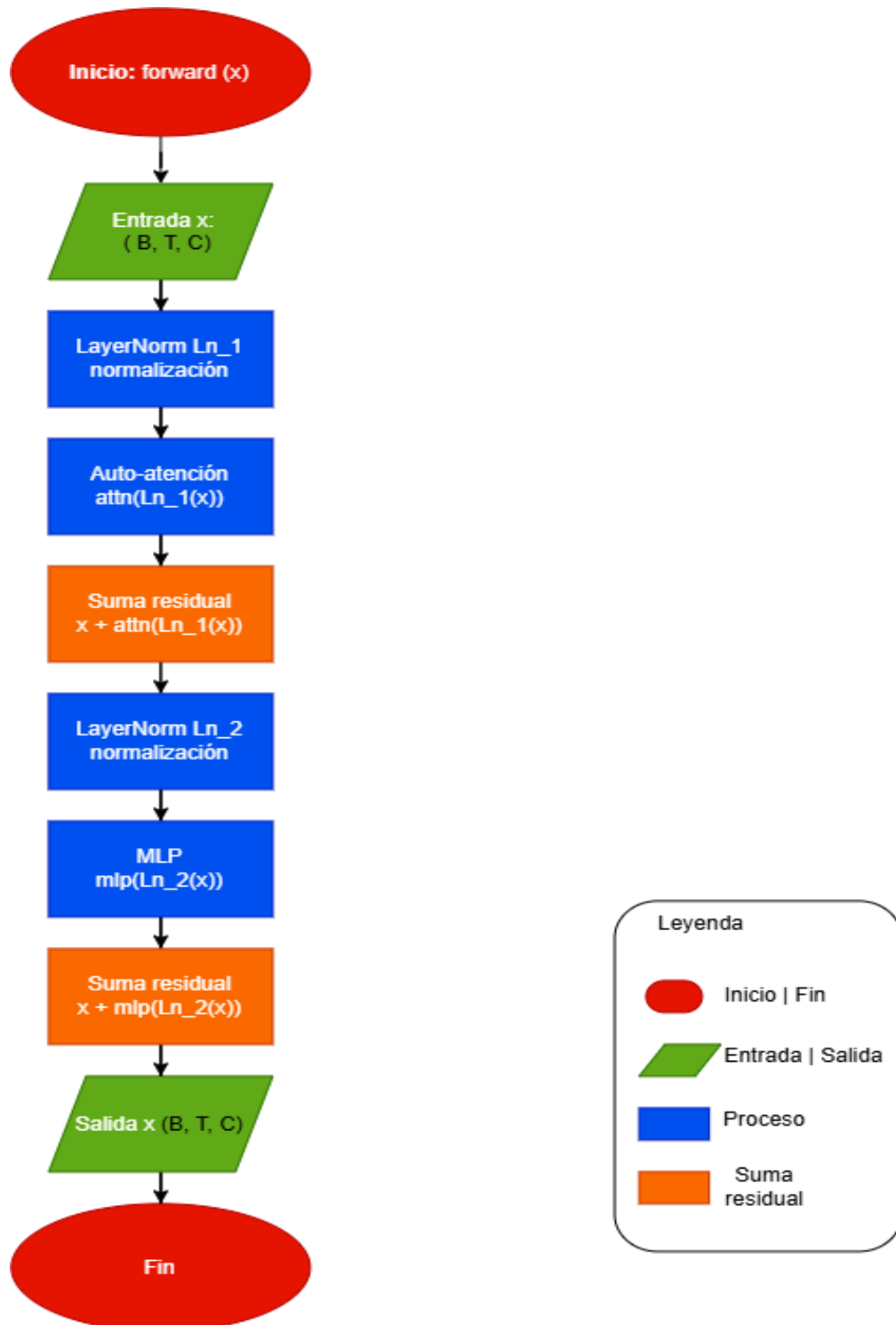
La función `__init__(self, config)`: Inicializa los componentes del bloque usando, creando las capas de normalización, atención y la red MLP.

- **`ln_1 (1capa)`:** Normaliza la información antes de la atención para estabilizar el procesamiento.
- **`attn (Auto Atención)`:** Analiza la relación entre los tokens anteriores para obtener contexto sin acceder a información futura.
- **`ln_2 (2capa)`:** Normaliza nuevamente los datos antes de la siguiente transformación.
- **`mlp (Perceptrón Multicapa)`:** Refina la información mediante transformaciones no lineales para aprender patrones complejos.

La función `forward (self, x)`: Define el flujo de procesamiento del bloque. Primero aplica la normalización y la atención, suma el resultado mediante una conexión residual y posteriormente repite el proceso con la red MLP.

A continuación, se presenta un diagrama de flujo que visualiza la operación de esta clase:

2.8 DIAGRAMA DE FLUJO (BLOCK):



2.9 Clase GPT (config) configuración:

Define la configuración básica del modelo GPT. En ella se establecen los principales parámetros utilizados para construir la arquitectura.

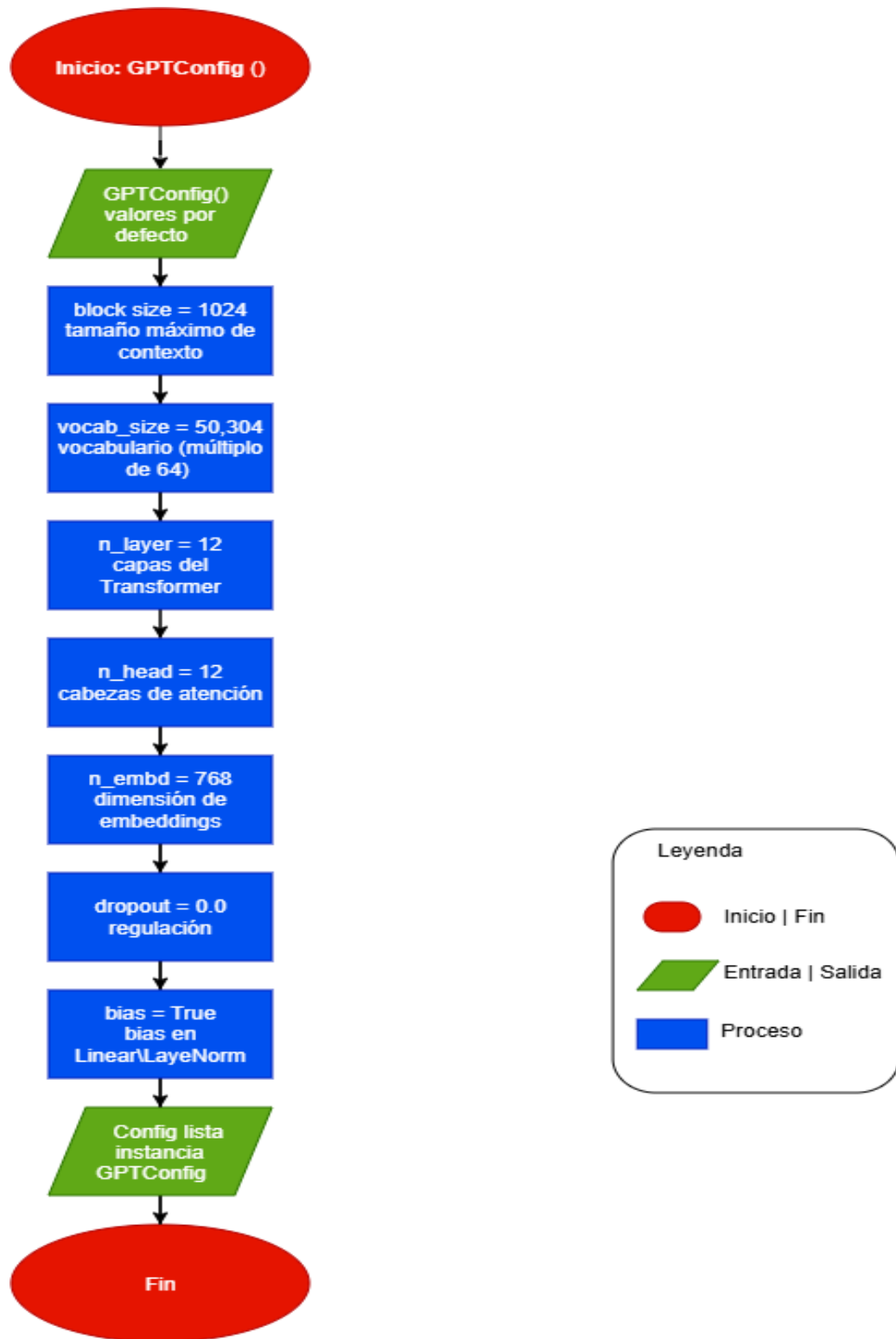
Se compone de los siguientes parámetros:

- **block_size:** la longitud máxima de la secuencia de entrada expresada en tokens (1024).
- **vocab_size:** el tamaño del vocabulario de tokens (50304).
- **n_layer:** el número de bloques Transformer (12).
- **n_head:** la cantidad de cabezas de atención (12).
- **n_embd:** la dimensión de los embeddings, los cuales representan cada token mediante vectores numéricos que posteriormente son procesados por las diferentes cabezas de atención para explorar múltiples perspectivas de la información del contexto.
- **dropout:** en este caso no está activo, con el propósito de que el modelo pueda ser entrenado a gran escala su valor es = 0.

Finalmente especifica si las capas lineales y LayerNorm utilizarán el término de sesgo (bias).

A continuación, se presenta un diagrama de flujo que visualiza la operación de esta clase:

2.10 DIAGRAMA DE FLUJO (GPT CONFIG):



2.11 Clase GPT (nn.Modulo):

Es el componente principal del modelo, ya que integra y coordina todas las clases anteriores para formar la arquitectura Transformer completa. Su función es gestionar el flujo de los datos desde la entrada de tokens hasta la predicción del siguiente token.

Dentro de esta clase se ejecutan estas funciones y elementos siguientes:

La función `__init__(self, config)`: Se inicializan los elementos del modelo.

- **wte (Token de palabras embebido):** Convierte los identificadores de los tokens en vectores numéricos.
- **wpe (Posicionamiento de palabra embebida):** Añade información sobre la posición de cada token dentro de la secuencia.
- **drop (apagado):** Reduce el sobreajuste desactivando aleatoriamente algunas neuronas durante el entrenamiento.
- **h (Bloques Transformer):** Contiene los bloques de atención y MLP que procesan la información contextual de los tokens.
- **ln_f (capa de normalización):** Normaliza la salida final para estabilizar el procesamiento.
- **lm_head:** Convierte la representación final de los tokens en probabilidades para predecir el siguiente elemento de la secuencia.

La función forward(): Ejecuta el procesamiento completo de GPT: embeddings → posiciones → bloques Transformer → salida de predicción.

La función generate(): Utiliza el modelo entrenado para generar texto token por token mediante predicción autoregresiva.

Ejemplo: Enunciado original: **Pedrito saludó a María.**

GPT predice (ayer).

Se reinicia el proceso: Pedrito saludó a María ayer

GPT predice (cuando).

Se reinicia el proceso: Pedrito saludó a María ayer cuando

GPT predice (ella).

Se reinicia el proceso: Pedrito saludó a María ayer cuando ella

GPT predice (estaba)

Se reinicia el proceso: Pedrito saludó a María ayer cuando ella estaba

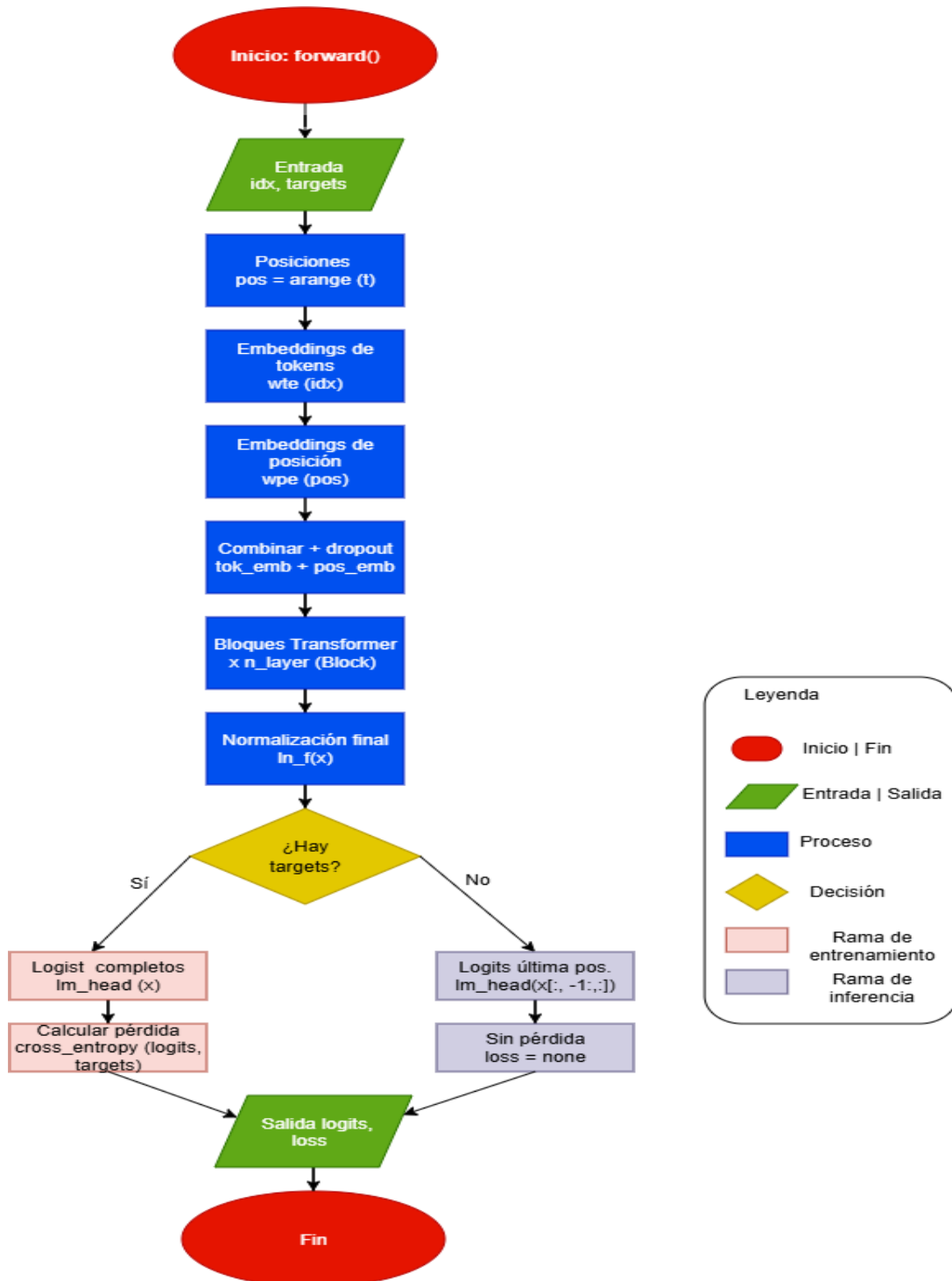
GPT predice (paseando).

Se reinicia el proceso: Pedrito saludó a María ayer cuando ella estaba paseando.

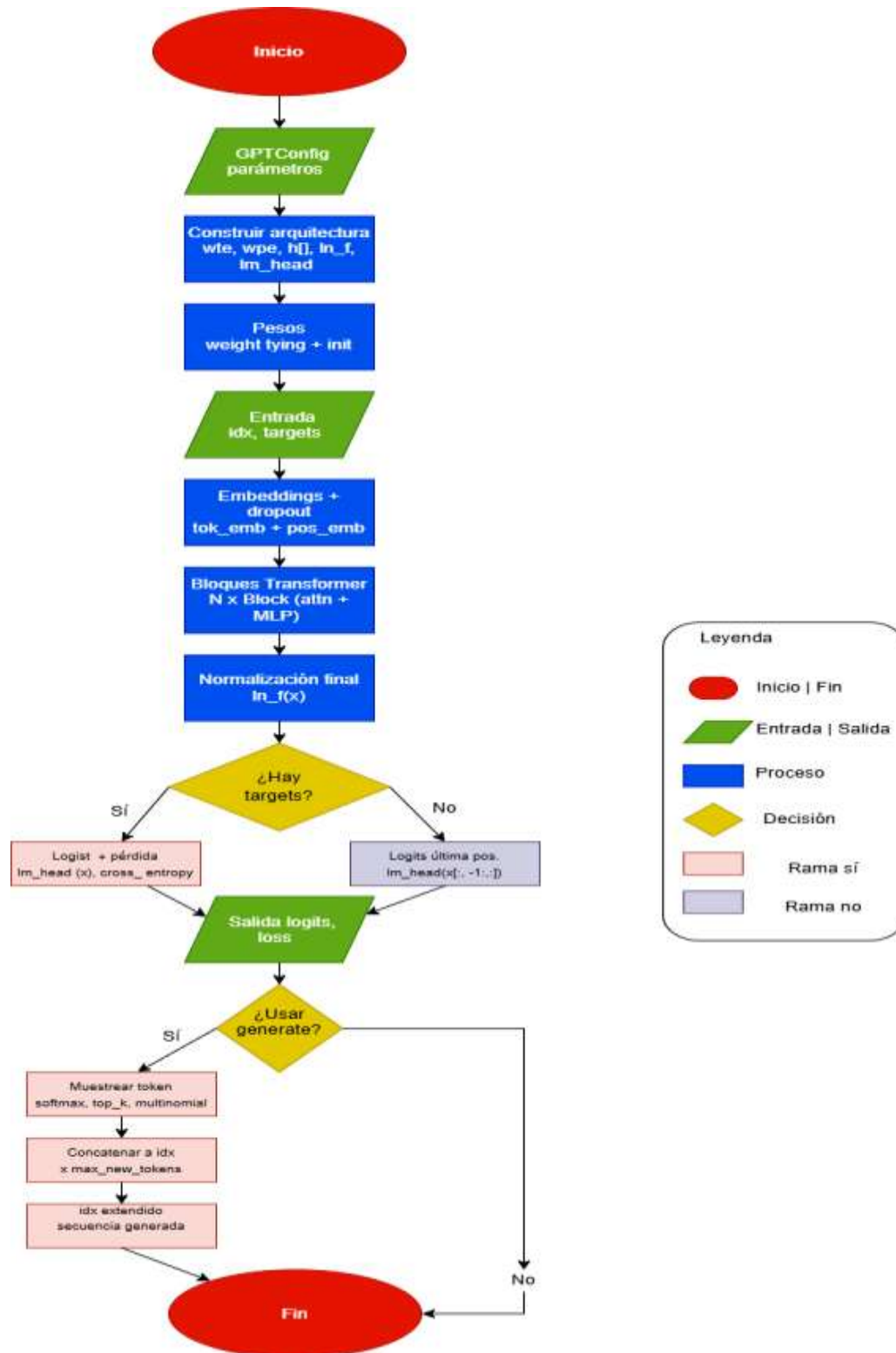
Y así sucesivamente, hasta elaborar construcciones textuales más complejas fundamentándose en la auto atención (que explora el sinnúmero de posibilidades contextuales para cada vector considerando a Q,K,V para saber cuál token es importante para otro); los perceptrones multicapa, que con esa información construyen una representación de token más exacta; para ello, expanden temporalmente el vector de 768 a un espacio de mayor dimensión, donde pueden aprender características más complejas, y luego lo reducen nuevamente a 768 dimensiones antes de enviarlo al siguiente bloque Transformer, que refina las representaciones uno por uno.

A continuación, se presenta un diagrama de flujo que visualiza la operación de esta clase:

2.12 DIAGRAMA DE FLUJO (GPT MODULE):



2.13 DIAGRAMA FLUJO COMPLETO DE “MODULE.PY” NANO GPT:



3. CICLO DE ENTRENAMIENTO DE APRENDIZAJE (TRAIN.PY).

3.1 Descripción.

Es el archivo que contiene la secuencia y los protocolos de aprendizaje que utiliza este modelo Transformer. Se encarga de cargar los datos, construir el modelo, entrenar, evaluar y guardar el resultado. En este caso, el modelo fue entrenado usando la versión de fábrica más pequeña, que es el documento en español de quijote_char.

Se utiliza como configuración de base los siguientes parámetros:

- **val_interval:** la frecuencia con que se autoevalúa (250 intentos).
- **eval_iters:** calcula los lotes para promediar la pérdida (200 lotes).
- **max_iters:** número de iteraciones totales (5000).
- **Batch_size:** número de secuencias simultáneas (64).
- **Block_size:** número de tokens en cada secuencia (256).
- **N_layer:** cantidad de bloques Transformers (6).
- **N_head:** cantidad de cabezas auto atención (6).
- **N_embd:** dimensiones de vectores de embebidos (384).
- **Learning_rate:** velocidad a la que ajustan sus pesos (1e-3).

Dentro de esta clase se ejecutan estas funciones y elementos siguientes:

La función def get_batch: toma el archivo principal de Quijote, el cual ha sido previamente convertido de letras a números en **prepare.py**, y posteriormente dividido en dos documentos:

- **train.bin:** concentra el 90% de los datos de Quijote y se usa como materia prima para entrenar a modelo.
- **val.bin:** representa el 10% de los datos y sirve para verificar que el modelo aprendió.

Parejas de entrenamiento de datos:

- **X:** se refiere a los datos de entrada (lo que el modelo recibe).
- **Y:** representa el siguiente token a predecir.

Consiste en que, Y siempre debe generar el siguiente token a partir de los datos de X, para ello se coloca 1 posición por delante. Matemáticamente $X=0$; $Y= y+1$. Repite este patrón sucesivamente porque Y es lo mismo que X pero adelantado.

Ejemplo: **Juan fue a...**

X: Juan fue a

Y: fue a visitar...

La función `if init_from == 'scratch'`: es la función que le indica al modelo que tiene que aprender desde 0 con los datos de entrenamiento suministrados (en este caso `quijote_char`); utilizando la configuración base que es la mencionada en la página anterior.

La función `estimate_loss()`: es la función que a modo de supervisión, analiza el entrenamiento del modelo para detectar si está aprendiendo o solo memorizando datos, usando como referencia a **`train.bin`** y **`val.bin`**. Si la primera baja y la segunda sube, entonces se considera overfitting. Se ejecuta gracias a **`@torch.no_grad()`**, heredado de Pytorch, que le permite calcular el error de entrenamiento sin guardar datos innecesarios que saturan la memoria.

Bucle principal del entrenamiento (while True:):

Se encarga de predecir, de calcular equivocaciones, de autocorregirse, y repetirse unas 5,000 veces (iteraciones totales). Funciona así:

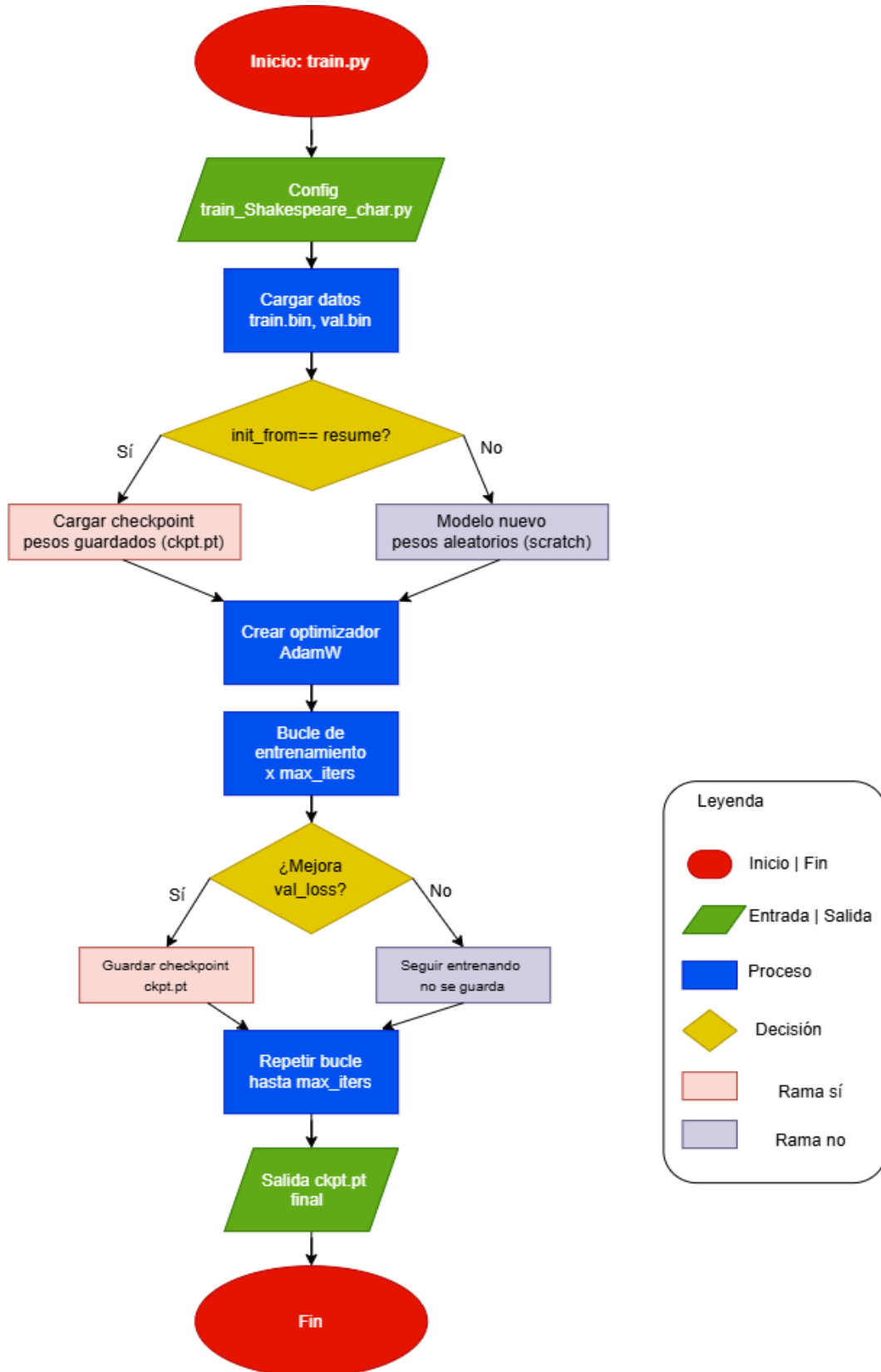
- **X, Y = get_batch('train'):** carga el batch de (X , Y).
- **lr = get_lr(iter_num):** ajusta la tasa de aprendizaje.
- **if iter_num % eval_interval == 0:** se autoevalúa para calcular pérdidas.
- **print(f"step {iter_num}: train loss:** proyecta en pantalla el proceso.
- **if:** guarda el punto de chequeo.
- **logits, loss = model(X, Y):** calcula el error;
- **(loss).backward():** hace retropropagación.
- **step(optimizer):** algoritmo AdamW optimiza los pesos.
- **X, Y = get_batch('train'):** se repite otra vez.

Guardado del modelo: ckpt.pt:

Funciona como el punto de control que cada 250 iteraciones, guarda todo lo aprendido por el modelo hasta ese momento. Se activa después de que **estimate_loss** haya calculado la pérdida. Funciona así:

- **if iter_num % eval_interval == 0:** cada 250 iteraciones, mide el error.
- **if losses['val'] < best_val_loss:** calcula si ha mejorado su aprendizaje.
- **best_val_loss = losses['val']:** guarda el récord de aprendizaje.
- **checkpoint = {:** agrupa pesos, optimizador, configuración, ajustes.
- **torch.save(checkpoint, os.path.join(out_dir, 'ckpt.pt')):** guarda.

3.1 DIAGRAMA FLUJO COMPLETO DE “TRAIN.PY” NANO GPT:



4. EL GENERADOR DE TEXTO (SAMPLE.PY).

4.1 Descripción:

Es el archivo que se encarga de tomar los datos que han sido entrenados por train.py y cargarlos en el modelo nanoGPT para generar texto comenzando desde el texto inicial, que en este caso es **quijote_char**.

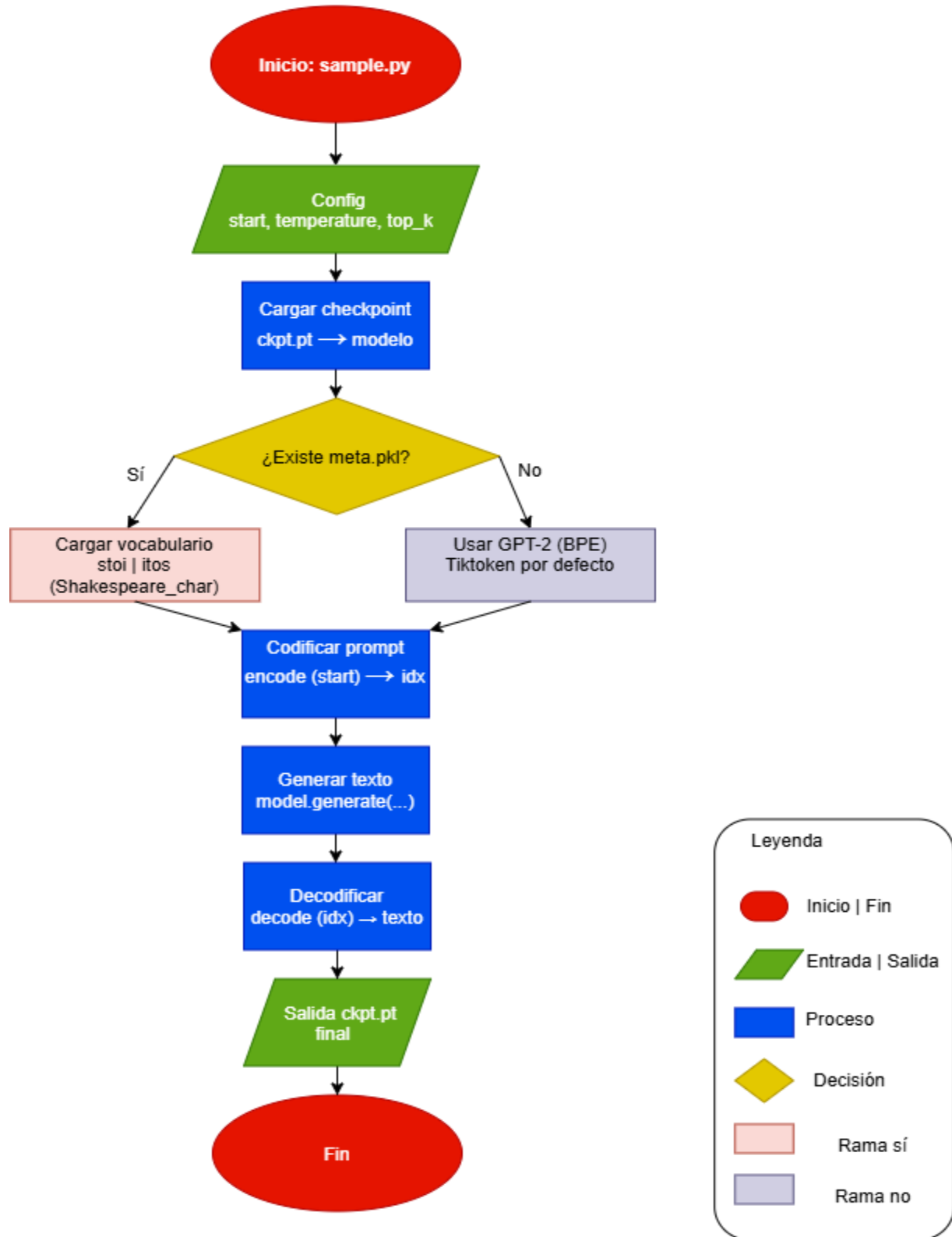
Dentro de este archivo se configuran estas variables siguientes:

- **init_from = 'resume':** carga un modelo previamente entrenado.
- **start:** texto inicial o prompt desde el que comienza la generación.
- **num_samples:** cantidad de textos que se generarán.
- **max_new_tokens:** cantidad máxima de caracteres que puede generar.
- **temperature:** controla el nivel de aleatoriedad.
- **top_k:** limita las opciones de caracteres que puede elegir el modelo.
- **device:** indica si se utiliza CPU o GPU.

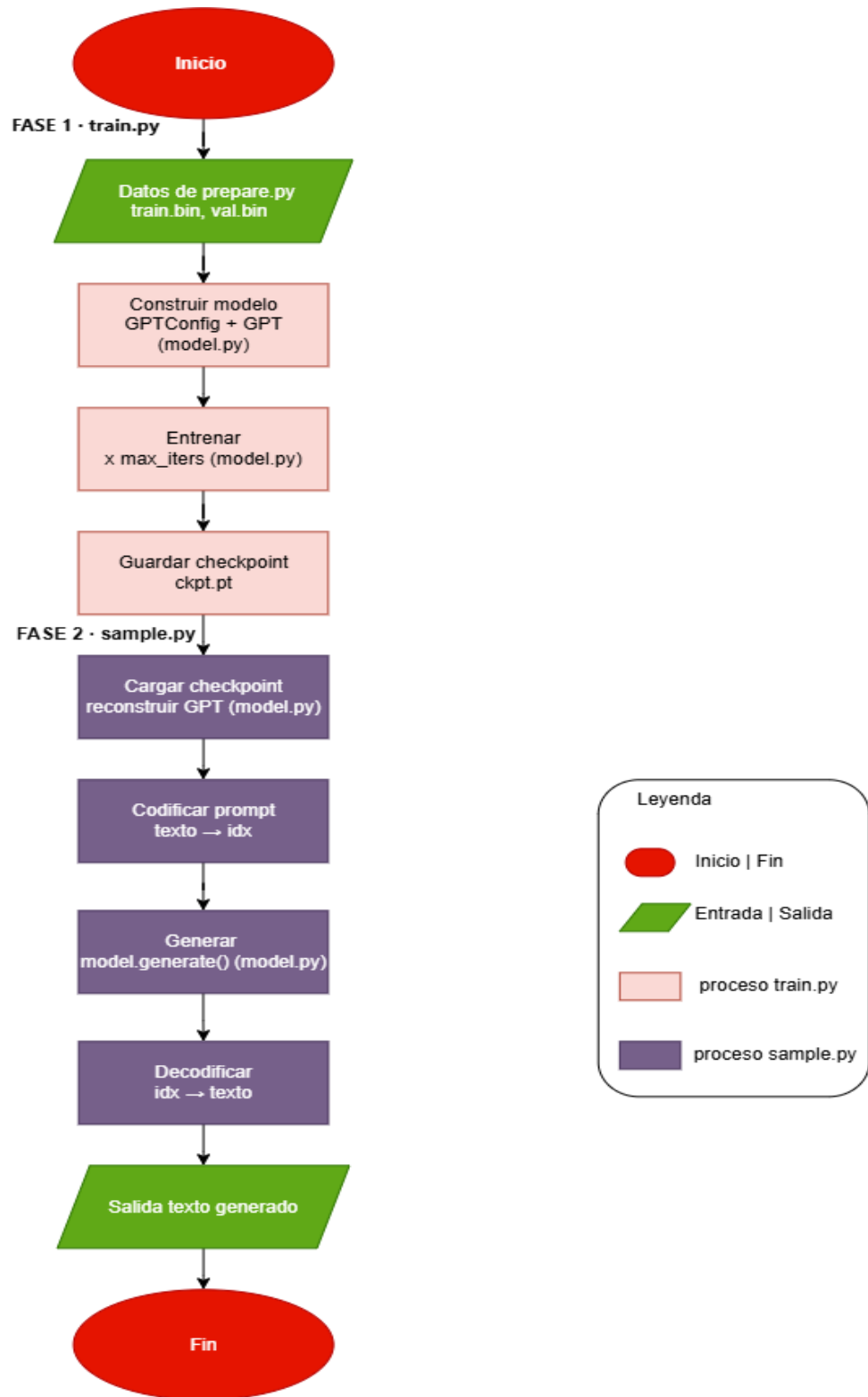
También se ejecutan estas condicionales:

- **ckpt_path = os.path.join(out_dir, 'ckpt.pt'):** Se encarga de busca los pesos y configuración del modelo ya entrenado.
- **with open(meta_path, 'rb') as f:** Aquí se carga el vocabulario al modelo para convertir entre caracteres (str) y númeos (int).
- **start = "\n":** frase de inicio que se le introduce al modelo para interactuar con él.
- **encode = lambda s: [stoi[c] for c in s]:** Esta es la Función dentro de la condicional que convierte el enunciado que introducimos en Start= “\n”, a número (encode)
- **x = (torch.tensor(start_ids, dtype=torch.long):** Tensor de Pytorch que organiza los números de stoi en lista de números para ser procesados.
- **model.generate(...):** Es la sección donde el modelo genera un carácter a la vez, una vez recibidos los caracteres (x), temperatura 0.8, el Tok K (los 200 tokens más probables según los logist).
- **decode = lambda l: ''.join([itos[i] for i in l]):** Función que se encarga de que la generación pase de números a letras que se pueden leer (decode).

4.1 DIAGRAMA FLUJO COMPLETO DE “SAMPLE.PY” NANO GPT:



5. DIAGRAMA DE NANOGPT (MODEL.PY, TRAIN.PY, SAMPLE.PY).



6. MODIFICACIONES DE VISUALIZACIÓN.

Con el fin de entender el funcionamiento interno del corazón del Transformer, así como la distribución de los logits y sus respectivas probabilidades; se decidió implementar dos scripts orientados a visualizar lo que ocurre en las profundidades del modelo.

1. `sample.py` — visualiza logits y softmax

Objetivo: ver cómo el modelo elige cada carácter, candidato por candidato, mientras genera texto.

Cómo evita tocar `model.py`: en vez de usar el método `model.generate()` que ya viene definido ahí, este script reescribe ese mismo bucle por fuera, en una función nueva (`generate_visual()`) dentro de `sample.py`. La clave está en esta línea:

python

`logits, _ = model(idx_cond)`

Esto llama al modelo como una "caja negra" — exactamente la misma llamada que `generate()` hace internamente en `model.py`, pero ejecutada desde afuera. Como nunca se edita la clase GPT ni sus métodos, `model.py` queda intacto. Lo único que se agregó fue:

Capturar probs (después del softmax) con `torch.topk()` para mostrar los candidatos más probables.

Un `print()` antes y después de cada elección, para que se vea "escribirse" el texto en vivo.

2. visualizar_atencion.py — visualiza las 6 cabezas + logits

Objetivo: ver el heatmap de atención de las 6 cabezas actualizándose token por token, además de los mismos logits/probabilidades.

Cómo evita tocar model.py: usa una técnica distinta pero igual de "no invasiva": `register_forward_hook()`. Esto es una función nativa de PyTorch que le permite "espiar" la entrada de cualquier capa sin modificar su código:

python

`block.attn.register_forward_hook(hacer_hook(i))`

El modelo corre exactamente como fue entrenado; el hook solo copia el tensor de entrada a un diccionario aparte (capturado). Después, la función `calcular_atencion()` recalcula la matriz de atención por fuera, usando esos datos capturados — es un "espejo" externo, nunca se altera **CausalSelfAttention** en sí. Sobre eso se agregó el mismo tipo de print de logits que tiene `sample.py`, y al final, la animación con `matplotlib.animation` guardada como GIF.

En ambos casos: `train.py` no se toca en absoluto — ninguno de los dos scripts participa del entrenamiento, solo cargan un checkpoint ya entrenado (`ckpt.pt`) y lo usan en modo inferencia (`model.eval()`).

3. DOCUMENTACIÓN DESCRIPTIVA DEL PROYECTO.

Aspecto técnico	Descripción del producto: model.py nanoGPT
Nombre del producto	nanoGPT – Implementación educativa de un modelo GPT basado en arquitectura Transformer.
Archivos analizados	model.py, train.py, sample.py y visualizar.py: archivos que contienen la definición de la arquitectura neuronal GPT completa, su entrenamiento, generación de texto y posterior visualización de logits, softmax y multiatención causal.
Tipo de solución tecnológica	Software de Inteligencia Artificial: red neuronal basada en arquitectura Transformer para Procesamiento del Lenguaje Natural (PLN).
Autor del proyecto	Andrej Karpathy, ingeniero e investigador en inteligencia artificial.
Lenguaje de programación	Python. Utiliza programación orientada a objetos mediante clases y módulos.
Versión recomendada de Python	Python 3.8 o superior (compatible con versiones actuales de Python 3.x).
Framework de IA utilizado	PyTorch, biblioteca de aprendizaje profundo para construcción y entrenamiento de redes neuronales.
Bibliotecas principales utilizadas	torch, torch.nn, torch.nn.functional, dataclasses, math, inspect y Matplotlib.
Arquitectura neuronal implementada	GPT (Generative Pre-trained Transformer), basada en bloques Transformer tipo Decoder.
Componentes principales del modelo	LayerNorm, CausalSelfAttention, MLP, Block, GPTConfig y GPT.

Área de aplicación	Procesamiento del Lenguaje Natural (PLN), generación de texto y modelos grandes de lenguaje (LLMs).
Funcionamiento principal	Recibe secuencias de tokens, procesa relaciones entre palabras mediante atención causal y predice el siguiente token.
Plataformas de ejecución	Computadoras personales, servidores, estaciones de trabajo con GPU y plataformas de computación en la nube.
Sistemas operativos compatibles	Linux (recomendado), Windows y macOS.
Aceleración de hardware	Compatible con GPU NVIDIA mediante CUDA y optimizaciones como Flash Attention (PyTorch 2.0+).
Requerimientos de hardware	Puede ejecutarse en CPU; para entrenamiento eficiente se recomienda GPU con memoria suficiente.
Base de datos utilizada	No utiliza bases de datos tradicionales como MySQL o PostgreSQL.
Manejo de datos	Trabaja con archivos de texto, conjuntos de datos lingüísticos, tokens y tensores procesados por PyTorch.
Manejo de archivos del modelo	Utiliza archivos de pesos o checkpoints mediante state_dict() para guardar y cargar parámetros aprendidos.
Entrada del modelo	Secuencias de tokens representadas numéricamente mediante embeddings.
Salida del modelo	Logits que representan la probabilidad de los siguientes tokens dentro del vocabulario.
Generación de texto	Mediante la función generate(), utilizando predicción autoregresiva token por token.
Optimización del entrenamiento	Utiliza el optimizador AdamW implementado en PyTorch.

Referencias de implementación	Basado en conceptos e implementaciones de GPT-2 de OpenAI y Hugging Face Transformers.
Licencia / disponibilidad	Proyecto de código abierto disponible para estudio, modificación y ejecución.

CONCLUSIÓN

Luego de haber conocido paso a paso el funcionamiento de la arquitectura neuronal Transformer, en este código nanoGPT de su autor Andrej Karpathy, se ha demostrado la eficiencia y el avance que ha significado para la tecnología el desarrollo del modelo original Transformer, pues al tener la capacidad de entender el nivel de relación entre los tokens que componen un enunciado sin olvidar las palabras iniciales, gracias a la **Auto Atención**; ha significado un mejoramiento exponencial del procesamiento del lenguaje natural y por ende, en el procesamiento de respuestas automatizadas que los grandes modelos de lenguaje pueden generar.

En este modelo, se apreció cómo a medida que fue entrenado por el texto en inglés de base Shakespeare_char, el Transformer fue reproduciendo palabras por palabras cada vez más coherentes en una clara demostración de la mecánica Auto Atención.

Ejemplo contundente del impacto que ha tenido esta tecnología es que hoy en día los modelos más avanzados como: GPT, Claude, DeepSeek, Kimi, están inspirados en Transformers, aunque mejorados con MoE (mezcla de expertos, para mayor eficiencia). Aun las futuras arquitecturas que puedan reemplazar a los Transformers; los Modelos de Espacio de Estado SSM (MAMBA), tendrán cosas en común con ellos. Por demás, este tipo de arquitectura ha servido de base para la Medicina, pues softwares como AlphaFold se ha destacado en el descubrimiento de nuevos fármacos; también en la conducción de vehículos autónomos pues las Vision Transformers han sustituido gradualmente a las Redes Neuronales Convolucionales.

Con todo esto, es evidente que estudiar este código fuente permite conocer de forma directa el funcionamiento de la arquitectura que soporta la Inteligencia Digital moderna y provee conocimiento fundacional para la que lo seguirá haciendo por hasta al menos los próximos 10-15 años.

Bibliografía

Las fuentes consultadas para la realización de esta investigación fueron:

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30. <https://arxiv.org/abs/1706.03762>
2. Karpathy, A. (2023, 16 de enero). Let's build GPT: from scratch, in code, spelled out [Video]. YouTube. <https://www.youtube.com/watch?v=kCc8FmEb1nY>
3. **Puiggros, V.** [Transformers: Qué son, cómo funcionan y por qué lo revolucionaron todo](https://inteligenciartificial.me/wiki/transformers/#:~:text=Un%20Transformer%20es%20una%20arquitectura%20de%20red%20neuronal,de%20lenguaje%20natural%20%28NLP%29%20y%20el%20aprendizaje%20profundo.) Recuperado de: <https://inteligenciartificial.me/wiki/transformers/#:~:text=Un%20Transformer%20es%20una%20arquitectura%20de%20red%20neuronal,de%20lenguaje%20natural%20%28NLP%29%20y%20el%20aprendizaje%20profundo.>
4. Pérez, J. Transformers y modelos de atención: conociendo los modelos de atención y su aplicación en el modelo transformer# Recuperado de: <https://www.dlsi.ua.es/~japerez/materials/transformers/attention/#transformers-y-modelos-de-atencion-conociendo-los-modelos-de-atencion-y-su-aplicacion-en-el-modelo-transformer>
5. Sotaquirá, M. (2026) ¿Qué es la ATENCIÓN en las Redes Transformer? [video] Youtube: <https://www.bing.com/videos/riverview/relatedvideo?q=auto+atenci%c3%b3n+transformer&&mid=81AC4DC7EF1BDB2858EB81AC4DC7EF1BDB2858EB&churl=https%3a%2f%2fwww.youtube.com%2fchannel%2fUCFVF0MpD1lNdU5VL3Pz67Yw&FORM=VRDGAR>
6. Sanderson, G. (2024, 7 de abril). Attention in transformers, step-by-step | Deep Learning Chapter 6. [Video]. YouTube. <https://www.bing.com/search?q=3%20blue%201%20brown%20action%20transformer%20visualmente&qsn&form=QBRE&sp=-1&ghc=1&lq=0&pq=3%20blue%201%20brown%20action%20transformer%20visualmente&sc=12-45&sk=&cvid=97D66F9648AC4906B7ED328FC2C614C6>